COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	July 31, 2026	**DATE SUBMITTED:**	
NAME:	De Vera, Sophia	**INSTRUCTOR:**	Engr. QUEJADO, Jimlord

Hands-On Activity 4.1 STACKS

1. PROCEDURE OUTPUT

ILO A Procedure

SCREENSHOT

main.cpp

```cpp
#include <iostream>
#include <stack>

int main (){
    // Create an object:
    std::stack <char> stack1;

    // Stacks: Push, Pop, isEmpty, isFull, Top

    std::cout << "Is the stack empty? " <<stack1.empty() << std::endl;
    stack1.push('S');
    std::cout << "The current top of the stack is " <<stack1.top() << std::endl;
    stack1.push('O');
    std::cout << "The current top of the stack is " << stack1.top() << std::endl;
    stack1.push('P');
    std::cout << "The current top of the stack is " << stack1.top() << std::endl;
    stack1.push('H');
    std::cout << "The current top of the stack is " <<stack1.top() << std::endl;
    stack1.push('I');
    std::cout << "The current top of the stack is " <<stack1.top() << std::endl;
    stack1.push('A');

    return 0;
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
> cd "c:\Users\TIPQC\Documents\DSA CARDENAS\DSA CARDENAS\HOA4.1_DeVera\" ; if
Is the stack empty? 1
The current top of the stack is S
The current top of the stack is O
The current top of the stack is P
The current top of the stack is H
The current top of the stack is I
```

OBSERVATION ANALYSIS

- This first procedure tackles the basic functions of the stacking structure. It shows us how to create a simple stacking array and how it can be a helpful programming language to use in terms of creating a library where we can implement a “first in, last out” operation.

ILO B1 Procedure

SCREENSHOT

ILOB_main.cpp ● main.cpp

ILOB_main.cpp > Pop()

```cpp
#include <iostream>

// Global Variables:
#define maxSize 10
int stackArr [maxSize];
int top = -1, newData;

/* Function Prototypes:
    1. Top
    2. Push
    3. Pop
    4. isEmpty
    5. isFull
*/

bool isEmpty();
bool isFull();
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
========================
What number do you want to push?
3
========================
Stack operations:
[1] Push [2] Pop [3] isEmpty [4] isFull [5] Top
Enter your choice:
4
========================
Is the stack full? 0
========================
Stack operations:
[1] Push [2] Pop [3] isEmpty [4] isFull [5] Top
Enter your choice:
2
========================
Popping the top stack: 3
========================
Stack operations:
[1] Push [2] Pop [3] isEmpty [4] isFull [5] Top
Enter your choice:
5
========================
The top element is: 8
========================
```

OBSERVATION ANALYSIS

- In this procedure, the program is tasked to execute a stack with a maximum size of 10, as stated in the first lines of the code. It has the main functions of the stacking structures, such as push, pop, isEmpty, isFull, and top. These elements mainly displays a menu that

	repeatedly prompts the user to choose an operation and execute it depending of their respective functions. 1. **Push** to add an element 2. **Pop** to remove the top element 3. **isEmpty** to check if the stack has no objects 4. **isFull** to check if the stack has reached its maximum size; and, 5. **Top** to view the top element

ILO B2 Procedure

SCREENSHOT

Welcome | main.cpp | stackList.h

main.cpp

```cpp
#include <iostream>
#include "stackList.h"

int main(){

    std::cout << "      Pushing 'C' 'P' 'E' onto the stack:     " << std::endl;
    //pushing function
    push('C');
    push('P');
    push('E'); // the current top since FILO

    //displaying the current top
    Top<char>();
    displayAll<char>();
    std::cout << "      Popping all the elements:     " << std::endl;

    while(!isEmpty<char>()){
        std::cout << " Popped: " << pop<char>() << std::endl;
    }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
      Pushing 'C' 'P' 'E' onto the stack:
The top of the stack is: E
Displaying the Stack:
E
P
C


      Popping all the elements:
 Popped: E
 Popped: P
 Popped: C
      Testing the Enderflow:
Stack Underflow.
```

OBSERVATION ANALYSIS

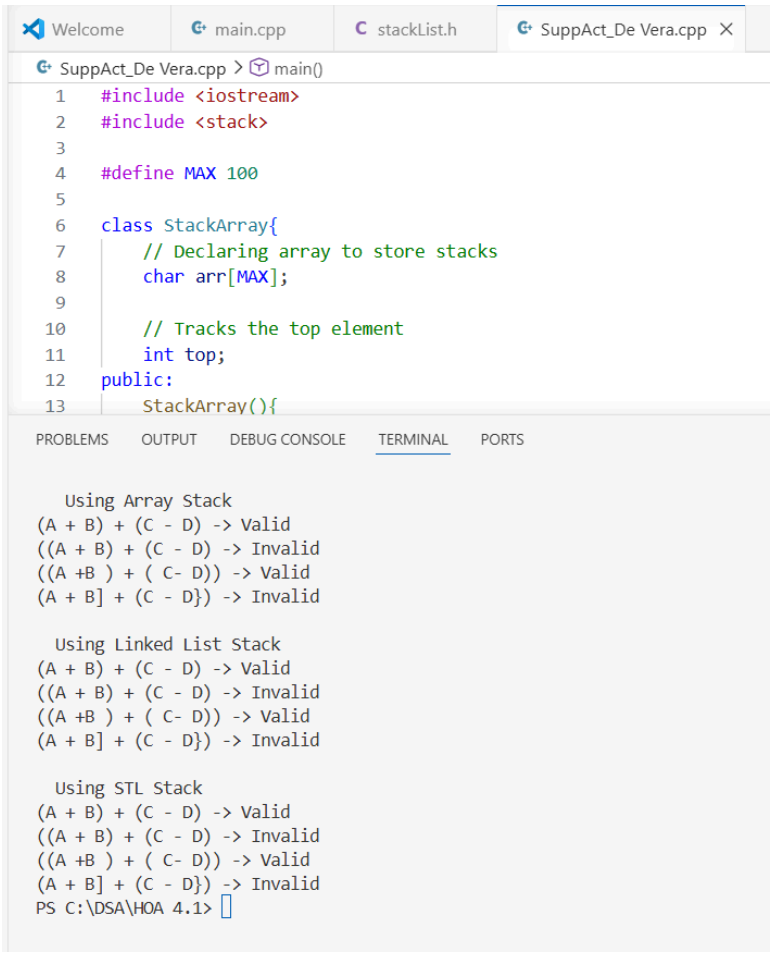
- The program successfully implemented a stack using a singly linked list, following the **Last In, First Out (LIFO)** principle. Wherein the letter C was the last to be printed out because it was the first to be encoded in the program and same goes with the letter E where it was the first to go out since it was the last to go in.

	- During the pop operations, the elements were removed in the reverse order **LIPF (LAST IN, POPPED FIRST).** - Overall, this kind of linked list implementation is very reliable when doing ***browser history*** since the last site used will be the first to be seen–and maybe deleted depending on the user.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SUPPLEMENTARY ACTIVTY

SCREENSHOT



OBSERVATION ANALYSIS	- As required from the instructions, we are tasked to check whether the given equations are balanced using the different stack implementations like **array-based**, **linked list-based**, and **STL.** Although the three have different methods, they all can come to the same conclusions/answers. - The program can also identify if there are some irregularities like wrong delimiters

	encoded of missing ones. - As i've finished from the ILOB.2, this supplementary activity is also a **Last In, First Out (LIFO).**

.

CONCLUSION:

- As i've finished this HOA about stacking and listing, I've come to learn that stacking is very important and is used in different ways. Like the **LAST IN, FIRST OUT** principle which I think was the main highlight of this HOA. As someone where coding is really essential and widely used, learning how stacks work can be a great use in solving real-world problems, such as checking balanced parentheses, brackets, and braces in expressions, which is commonly done in compilers and syntax checking.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Google Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Asking for clarifications.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.